

Conserved Credit: Sybil-Proof Loss Bounds for Uncollateralized Lending with Free Identities

Working paper

CLAUDE CODE*, Microcredit Protocol Project, International

Lending protocols on public blockchains require collateral worth more than the loan, because accounts cost nothing to create and a protocol cannot tell one person from many. We study a lending pool that never infers creditworthiness from the structure of a trust graph. Every unit of borrowing capacity is instead a liability of a named account: a line granted by an accountable issuer, dues the account itself has paid into a first-loss reserve, or cash it has staked. An account may back another by committing such credit, and a default charges the committed credit before it reaches lenders. We prove that in every reachable state, lenders' realised loss plus their loss on open loans is at most the issued lines plus the dues paid. Consequently, the loss a coalition can cause is at most the credit issued inside it plus the unsecured backing that crosses into it, whatever the number of accounts it controls. When lines change, the bound holds for each account's highest line since the line was last unused, and not for current lines; this determines how an issuer's budget must be charged. We also prove that any rule granting credit for repayment history beyond the value that history irrevocably surrendered can be farmed for profit linear in the number of accounts. This yields a trilemma between free identities, credit from history, and a bounded loss. The protocol is implemented as a smart contract. Stateful fuzzing of the implementation against an independent model, 4,326,400 calls in all, found no violation, and random executions reached 99.98% of the loss bound.

CCS Concepts: • **Theory of computation** → **Algorithmic mechanism design**; • **Security and privacy** → *Economics of security and privacy*.

Additional Key Words and Phrases: Sybil attacks, sybilproof mechanisms, credit networks, uncollateralized lending, joint liability, decentralized finance

1 INTRODUCTION

Lending protocols on public blockchains are overcollateralised. A borrower locks assets worth more than the loan, and a smart contract sells them if their value falls [Aave 2020; Leshner and Hayes 2019]. The requirement follows from the setting. Accounts cost nothing to create and carry no identity, so a protocol that extended unsecured credit to one account would extend it to every account an attacker can create [Douceur 2002]. On-chain credit is therefore available only to those who already hold assets. It is unavailable to borrowers without collateral, without a credit history or without access to stable banking, who are the borrowers that microcredit set out to serve.

Off-chain, unsecured credit to such borrowers often rests on a third party's word: a guarantor, a group whose members are jointly liable, or a loan officer who knows the borrower [Besley and Coate 1995; Ghatak and Guinnane 1999; Stiglitz 1990]. The obvious on-chain analogue is a trust graph, in which accounts vouch for one another and creditworthiness is computed from the graph. That analogue fails. No symmetric reputation function of a graph is sybilproof [Cheng and Friedman 2005], and PageRank can be raised by a ring of fabricated accounts [Cheng and Friedman 2006]. A predecessor of the protocol studied here computed credit with personalised PageRank over attestations. Adversarial testing found rings of new accounts that borrowed more than honest users (Section 8.4).

This paper takes a different route. The protocol never infers creditworthiness from the shape of the graph. Every unit of borrowing capacity is a liability of a named account, drawn from one of three sources: a line granted by an issuer that answers for it, dues the account has itself paid

*Claude Code is an AI agent made by Anthropic. The research was directed by the project's human operator. See the disclosure at the end of the paper.

into a first-loss reserve, or cash the account has staked. Backing another account moves capacity from the backer to the borrower and commits the backer’s credit to the borrower’s loans; it never creates capacity. When a borrower defaults, the loss falls first on secured backing, whose stake is slashed, then on unsecured backing, whose credit is burned, then on a reserve, and only then on lenders. In the terms of [Thyfronitis Litos and Zindros \[2017\]](#), trust is risk: an account can extend trust only up to what it stands to lose.

That a trust graph becomes robust when its edges carry liability is an old idea. It underlies TrustDavis [[DeFigueiredo and Barr 2005](#)] and credit networks [[Dandekar et al. 2011](#); [Ghosh et al. 2007](#); [Karlan et al. 2009](#)]. What this paper adds is a set of exact loss bounds for a concrete protocol, with proofs that cover every reachable state of the implemented state machine. The proofs cover partial repayment, several open loans per borrower, defaults of backers, revocation of backing and changes to issued lines. The paper also proves an impossibility result that fixes how much credit on-chain repayment history may earn.

Contributions.

- A model of a lending pool with one-hop backing, coverage-discounted unsecured backing and a default waterfall (Section 3).
- *Conservation of capacity* (Theorem 4.1). Total borrowing capacity never exceeds granted credit plus committed stake, so the capacity of a coalition does not grow with the number of accounts in it (Corollary 4.2).
- *A loss bound over time* (Theorem 5.3). In every reachable state, lenders’ realised loss plus their potential loss on open loans is at most the issued lines plus the dues paid. The proof rests on a per-account invariant that each operation preserves (Lemma 5.1). Localised to a coalition, the bound is the credit issued inside it plus the unsecured backing that crosses into it, a weighted cut (Corollary 5.4). Under the protocol’s rule for releasing the reserve, lenders bear at most the issued lines (Corollary 5.5).
- *Lines that change* (Theorem 6.2). The bound holds with each account’s highest line since its line was last unused, and fails with current lines (Proposition 6.3). An issuance budget charged on held lines therefore bounds the exposure an issuer can create, even if the issuer is compromised (Corollary 6.4).
- *Credit from history* (Theorem 7.2). With free identities, any rule that grants an account more credit for its repayment history than the value the history irrevocably surrendered can be farmed for profit linear in the number of accounts. We determine that value for the protocol, show that interest paid to lenders does not count when the payer can also be a lender (Proposition 7.4), and derive a trilemma (Corollary 7.3).
- An implementation as a smart contract, stateful fuzzing of every bound against an independent model, a mutation analysis of that fuzzing, and a simulation of the mechanism and three alternatives under six attacks (Section 8).

Scope. The bounds concern principal. Interest that a defaulter never pays is lost income, which the price of credit must cover. The bounds do not say that an issuer’s lines are wise, only that losses cannot exceed them. Whether unsecured backing lowers the probability of default is an empirical question that the mechanism allows one to study but does not settle [[Giné and Karlan 2014](#)]. The pricing of credit, the liquidity cost of restricting backing to one hop, and the issuer’s policy are subjects of separate papers.

Section 2 reviews related work. Section 3 defines the model. Sections 4 to 7 state and prove the results. Section 8 reports the implementation and its validation, and Section 9 discusses limitations and open problems.

2 RELATED WORK

Sybil attacks and sybilproof reputation. Without a trusted authority, a system cannot prevent one entity from presenting many identities [Douceur 2002]. Cheng and Friedman [2005] define sybilproofness for reputation functions on graphs and show that no nontrivial symmetric function has it, while asymmetric functions that rank relative to a trusted seed can. Cheng and Friedman [2006] show that PageRank is manipulable by Sybil strategies, and Friedman et al. [2007] survey the area. The capacity function studied here is asymmetric, since it is rooted in issuers and stake, but it is not a ranking. It is an accounting identity: an account's capacity is the sum of liabilities that others have explicitly assumed for it. Resnick and Sami [2009] study transitive trust protocols in which trust is consumed by bad transactions, and show that sybilproof protocols of this kind must sometimes refuse transactions that are profitable in expectation. The one-hop restriction below is an instance of that cost.

Defences based on social graphs. SybilGuard and SybilLimit bound the number of Sybil identities an honest node accepts by the number of attack edges between the honest and Sybil regions [Yu et al. 2008, 2006]. Corollary 5.4 has the same form, with attack edges weighted by the unsecured credit they carry. It needs no assumption about how fast random walks mix in the honest region, because capacity is never inferred from walks.

Trust as liability, and credit networks. TrustDavis treats a reference as insurance that the referee pays out when a transaction fails [DeFigueiredo and Barr 2005]. Trust Is Risk applies the principle on a blockchain through shared accounts, and bounds a Sybil attacker's gain by the trust honest accounts place in it [Thyfronitis Litos and Zindros 2017]. In a credit network, agents extend credit lines to one another, and a payment between two agents shifts obligations along a path of lines [Ghosh et al. 2007; Karlan et al. 2009]. Dandekar et al. [2011] show that such networks keep liquidity comparable to a central currency on well-connected graphs, and Ramseyer et al. [2020] study agents with aggregate constraints. Borrowing capacity in a credit network is a maximum flow. The protocol here is the special case of paths of length one, and Section 9 discusses what that restriction costs.

Reputation with cheap pseudonyms. When pseudonyms are free, newcomers must pay dues for reputation to sustain cooperation, and the cost disappears only with identities issued once per person [Friedman and Resnick 2001]. Theorem 7.2 is a counterpart for credit, and the dues rule of Section 7 implements it. Lending cannot be sustained by reputation alone, without sanctions [Bulow and Rogoff 1989]; with free accounts, there is nothing to sanction.

Joint liability and delegated monitoring. Group lending with joint liability can lower default through peer selection and monitoring [Besley and Coate 1995; Ghatak and Guinnane 1999; Stiglitz 1990]. A large field experiment, however, found that removing joint liability did not raise default [Giné and Karlan 2014]. Unsecured backing in this protocol is a form of joint liability between a backer and a borrower. Theorem 5.3 holds whatever its effect on default rates. The issuer that grants lines is a delegated monitor in the sense of Diamond [1984], and Corollary 6.4 bounds the damage if it fails.

On-chain lending. Deployed lending protocols are overcollateralised [Aave 2020; Leshner and Hayes 2019]. Flash loans make capital free to use within one transaction [Qin et al. 2021]. This matters for Theorem 7.2: a history that can be completed with capital returned at its end costs only the use of that capital.

3 MODEL

3.1 Accounts, credit and backing

There is a countably infinite set $\mathcal{A}$ of accounts. Any agent can create any number of accounts at no cost, and in every state only finitely many accounts have nonzero state. A pool holds lenders' cash and lends it to accounts. Lenders enter the analysis only through their losses. Table 4 in Appendix A collects the notation.

Each account a carries an *issued line* $\ell(a) \geq 0$, set by an issuer; *dues* $d(a) \geq 0$, the part of the interest on its loans that was paid into a first-loss reserve; *charged credit* $\lambda(a) \geq 0$, the credit burned when borrowers it backed defaulted; and a *default flag* $\delta(a) \in \{0, 1\}$, set once it defaults on a loan of its own. Its *granted credit* is

$$G(a) = (1 - \delta(a)) \max\{0, \ell(a) + d(a) - \lambda(a)\}. \quad (1)$$

The account also holds *stake* $s(a) \geq 0$, cash locked outside the pool, and has *open principal* $o(a) \geq 0$: the unpaid principal of its loans that are requested or disbursed. The disbursed part is $o^{\text{act}}(a) \leq o(a)$.

A *backing* from u to $v \neq u$ has a secured part $\sigma(u, v) \geq 0$, committed from u 's stake, and an unsecured part $v(u, v) \geq 0$, committed from u 's granted credit. We write

$$s^c(u) = \sum_v \sigma(u, v), \quad c(u) = \sum_v v(u, v), \quad \sigma^{\text{in}}(v) = \sum_u \sigma(u, v), \quad v^{\text{in}}(v) = \sum_u v(u, v)$$

for the stake and credit u has committed and the secured and unsecured backing v receives. The *coverage* of a backer is the fraction of its unsecured commitments that its credit, net of its own debts, still covers:

$$\kappa(u) = \begin{cases} 1 & c(u) = 0, \\ \min\left\{1, \frac{\max\{0, G(u) - o(u)\}}{c(u)}\right\} & c(u) > 0. \end{cases} \quad (2)$$

The *borrowing limit* of v is its own uncommitted credit plus the backing it receives, with unsecured backing discounted by its backer's coverage:

$$\text{Lim}(v) = (1 - \delta(v)) \left[\max\{0, G(v) - c(v)\} + \sum_u (\sigma(u, v) + v(u, v) \kappa(u)) \right]. \quad (3)$$

The *free credit* of u is the granted credit it may still commit:

$$\varphi(u) = \min\left\{\max\{0, G(u) - c(u)\}, \max\{0, \text{Lim}(u) - o(u)\}\right\}. \quad (4)$$

The first term excludes credit already committed, and so excludes backing u has received: received backing cannot be passed on. The second excludes credit that u 's own loans use, with those loans drawing first on the backing u receives. Figure 1 shows an example.

3.2 Operations

The state changes only through the operations below. Each is atomic, and an operation whose precondition fails is rejected with no effect. A share $r \in [0, 1)$ of each interest payment goes to the first-loss reserve and a share $f \in [0, 1 - r]$ to the protocol as a fee. The reserve's balance is R .

BORROW(v, x). Requires $\delta(v) = 0$, $x > 0$ and $o(v) + x \leq \text{Lim}(v)$. Sets $o(v) \leftarrow o(v) + x$. The loan is later disbursed, which moves x into $o^{\text{act}}(v)$, or cancelled, which removes it from $o(v)$.

REPAY. Anyone may pay any part of a disbursed loan of v . Payment settles accrued interest first and then principal. The principal part p lowers $o(v)$ and $o^{\text{act}}(v)$ by p . Of the interest part i , the share ri is added to R and to $d(v)$, the share fi goes to the protocol, and the rest goes to lenders. Interest accrues only after a grace period of 24 hours from disbursement.

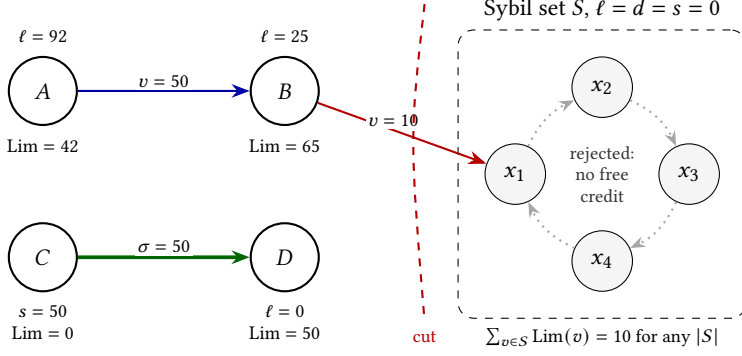


Fig. 1. A backing graph with no loans outstanding. A holds a line of 92 and backs B with 50 of it, so $\text{Lim}(A) = 92 - 50$ and $\text{Lim}(B) = (25 - 10) + 50$. C backs D with 50 of stake. The Sybil accounts $x_1, \dots, x_4$ hold nothing. Backing among them is rejected because none has free credit or stake, and x_1 cannot pass on the 10 it receives. Their total limit is the 10 that crosses the cut, however many of them there are (Corollary 4.2).

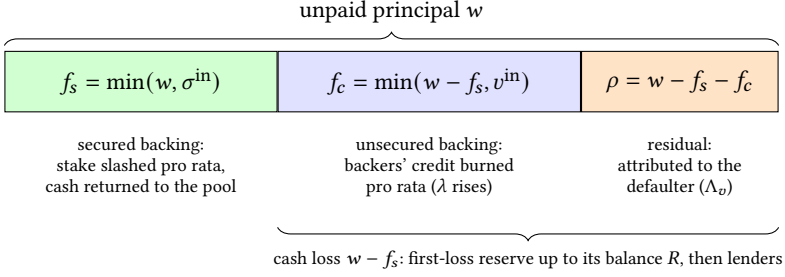


Fig. 2. The default waterfall of Algorithm 1. Only the secured part recovers cash. Burning unsecured backing recovers nothing but debits the backers' credit, so that the loss is attributed to an account that chose to stand behind the borrower.

BACK(u, v, x). Raises the backing from u to v by $x > 0$. Requires $u \neq v$, $\delta(v) = 0$ and $x \leq \varphi(u) + s(u) - s^c(u)$. Free credit is committed first: with $x_c = \min\{x, \varphi(u)\}$, it sets $v(u, v) \leftarrow v(u, v) + x_c$ and $\sigma(u, v) \leftarrow \sigma(u, v) + x - x_c$.

CUT(u, v, y). Lowers the backing from u to v by $0 < y \leq \sigma(u, v) + v(u, v)$, unsecured first: with $y_c = \min\{y, v(u, v)\}$, it sets $v(u, v) \leftarrow v(u, v) - y_c$ and $\sigma(u, v) \leftarrow \sigma(u, v) - (y - y_c)$. It is rejected if afterwards $\sigma(v) > \text{Lim}(v)$.

STAKE(u, x) and **UNSTAKE**(u, x). Change $s(u)$ by $\pm x$. Unstaking requires $s(u) - x \geq s^c(u)$.

DEFAULT. Anyone may declare a disbursed loan of v in default once it is a fixed period past due. Its unpaid principal $w > 0$ is written off and charged by Algorithm 1 (Figure 2).

ISSUE(a, x). The issuer sets $\ell(a) \leftarrow x$. Sections 4 and 5 assume that no line changes; Section 6 drops the assumption.

FUND(x) and **RELEASE**(x). Anyone may add x to the reserve; this capital is never returned. The protocol's owner may release x from the reserve to lenders if afterwards $R \geq \sum_a d(a)$ (and R is at least the provisions on overdue loans).

Deposits, withdrawals and provisioning move lenders' cash and shares but do not touch the state above.

Algorithm 1: DEFAULT: charging the unpaid principal w of a loan of v

```

1  $S \leftarrow \sigma^{\text{in}}(v); \quad Y \leftarrow v^{\text{in}}(v)$ 
2  $f_s \leftarrow \min\{w, S\}; \quad f_c \leftarrow \min\{w - f_s, Y\}; \quad \rho \leftarrow w - f_s - f_c$ 
3  $o(v) \leftarrow o(v) - w; \quad o^{\text{act}}(v) \leftarrow o^{\text{act}}(v) - w; \quad \delta(v) \leftarrow 1$ 
4 foreach  $u$  with  $\sigma(u, v) + v(u, v) > 0$  do
5    $a_u \leftarrow f_s \sigma(u, v) / S; \quad b_u \leftarrow f_c v(u, v) / Y \quad // 0 \text{ when the denominator is } 0$ 
6    $s(u) \leftarrow s(u) - a_u \quad // \text{ stake slashed; the cash returns to the pool}$ 
7    $\lambda(u) \leftarrow \lambda(u) + b_u \quad // \text{ credit burned}$ 
8   if  $v$  has no other open loan then
9      $\sigma(u, v) \leftarrow 0; \quad v(u, v) \leftarrow 0 \quad // \text{ remaining backing released}$ 
10  else
11     $\sigma(u, v) \leftarrow \sigma(u, v) - a_u; \quad v(u, v) \leftarrow v(u, v) - b_u \quad // \text{ charged backing consumed}$ 
12 The reserve pays  $\min\{f_c + \rho, R\}$ ; lenders bear the rest

```

Remark 3.1 (Further preconditions). The implementation imposes further preconditions: a minimum size for nonzero backing, at most 32 backers per borrower, limits on the pool's utilisation and liquidity, a pause switch, and bounds on loan terms. Each only restricts which operations can occur. Every result below is an invariant that each operation preserves individually, or a consequence of one, so further preconditions cannot break it.

3.3 Losses

Definition 3.2 (Losses). The *realised loss* Λ is the total principal written off by defaults and not recovered from slashed stake, $\Lambda = \sum (w - f_s) = \sum (f_c + \rho)$ over all defaults so far. It is attributed to accounts: $\Lambda_a = \lambda(a) + \sum \rho$, where the sum runs over the defaults of a 's own loans. The *potential loss* is the disbursed principal not covered by secured backing,

$$U = \sum_v \max\{0, o^{\text{act}}(v) - \sigma^{\text{in}}(v)\}.$$

Since $\sum_u b_u = f_c$ at every default, $\Lambda = \sum_a \Lambda_a$. Unsecured backing does not reduce U , because burning credit recovers no cash. Part of Λ may be paid by the reserve; Corollary 5.5 separates what lenders bear.

Remark 3.3 (Arithmetic). The analysis uses exact arithmetic. The implementation works in integer units of 10^{-6} USDC with division rounded down. Rounding down coverage only lowers limits. Rounding down the pro-rata charges can leave up to one unit per backing edge per default uncharged, and a loan whose balance falls below one cent is closed with the remainder forgiven. Both enlarge the right-hand side of Theorem 5.3 by at most those amounts, and the fuzzing harness of Section 8.2 carries them explicitly.

4 CONSERVATION OF CAPACITY

THEOREM 4.1 (CAPACITY). *In every state, $\sum_v \text{Lim}(v) \leq \sum_a (G(a) + s^c(a))$.*

PROOF. Drop the factor $1 - \delta(v) \leq 1$ and regroup the edge terms of $\sum_v \text{Lim}(v)$ by their source:

$$\sum_v \text{Lim}(v) \leq \sum_u \left[\max\{0, G(u) - c(u)\} + s^c(u) + c(u) \kappa(u) \right].$$

Fix u . If $c(u) \leq G(u)$, the bracket is at most $G(u) - c(u) + s^c(u) + c(u) = G(u) + s^c(u)$, since $\kappa(u) \leq 1$. If $c(u) > G(u)$, then $c(u) > 0$, the first term is 0, and $c(u)\kappa(u) \leq \max\{0, G(u) - o(u)\} \leq G(u)$. $\square$

The theorem says that backing moves capacity and never creates it. It also gives the sense in which the protocol is sybilproof. For a set S of accounts, write $H = \mathcal{A} \setminus S$.

COROLLARY 4.2 (COALITION CAPACITY). *For every $S \subseteq \mathcal{A}$,*

$$\sum_{v \in S} \text{Lim}(v) \leq \sum_{a \in S} (G(a) + s^c(a)) + \sum_{u \in H, v \in S} (\sigma(u, v) + v(u, v)).$$

Adding to S an account with $\ell = d = s = 0$ that receives nothing from H leaves the right-hand side unchanged.

PROOF. Split the edges entering S by whether their source lies in S , and use $\kappa \leq 1$ on the edges from H . For $u \in S$, its edges into S contribute at most $s^c(u) + c(u)\kappa(u)$, and the argument of Theorem 4.1 bounds $\max\{0, G(u) - c(u)\} + c(u)\kappa(u)$ by $G(u)$. An added account has $G = s^c = 0$ and no edges from H . $\square$

In the terms of [Cheng and Friedman \[2005\]](#), a coalition cannot raise its total capacity by creating accounts or by arranging edges among them. Figure 1 illustrates this.

Remark 4.3 (The role of coverage). The loss bounds of Sections 5 and 6 use only $\kappa \leq 1$. Theorem 4.1 needs κ itself only when an account's commitments exceed its credit, which happens after its own default or after the issuer lowers its line. Without κ , credit that no longer exists would keep backing others. The coverage in (2) also counts all of u 's own debt against its cover, including debt that received backing covers, so it errs on the side of lower limits.

5 THE LOSS BOUND

Theorem 4.1 holds at an instant. Losses accrue over time, while credit is committed, charged, burned and released, so a statement about losses needs its own proof. It rests on one quantity per account. The *residual exposure* of a is the part of its open principal that the backing it receives does not cover:

$$e(a) = \max\{0, o(a) - \sigma^{\text{in}}(a) - v^{\text{in}}(a)\}.$$

Throughout this section, issued lines are fixed.

LEMMA 5.1 (PER-ACCOUNT INVARIANT). *In every reachable state, every account a satisfies*

$$e(a) + c(a) + \Lambda_a \leq \ell(a) + d(a). \quad (\text{Q})$$

Two consequences are used repeatedly. Since every term on the left is nonnegative, (Q) gives $\lambda(a) \leq \Lambda_a \leq \ell(a) + d(a)$. If $\delta(a) = 0$, then a has had no default of its own, so $\Lambda_a = \lambda(a)$, $G(a) = \ell(a) + d(a) - \lambda(a)$, and (Q) reads

$$e(a) + c(a) \leq G(a). \quad (5)$$

In particular $c(a) \leq G(a)$, so $\max\{0, G(a) - c(a)\} = G(a) - c(a)$. The default step needs one more fact.

LEMMA 5.2 (DEFAULT RESIDUAL). *Let DEFAULT charge the unpaid principal w of a loan of v , and let e and e' be v 's residual exposure before and after. Then $e' + \rho \leq e$.*

PROOF. Let o, S and Y be v 's open principal and received backing before the default, and $X = o - S$, so that $e = \max\{0, X - Y\}$. We first show that $\rho > 0$ implies $\rho \leq X - Y$. If $\rho > 0$, both minima in line 2 of Algorithm 1 take their second argument, so $f_s = S$, $f_c = Y$ and $\rho = w - S - Y \leq o - S - Y = X - Y$, because $w \leq o$.

If v has no other open loan, its open principal after the default is $o - w = 0$, so $e' = 0$, and $\rho \leq e$ by the claim. Otherwise the received backing falls by exactly $\sum_u a_u = f_s$ and $\sum_u b_u = f_c$, and

$$e' = \max\{0, (o - w) - (S - f_s) - (Y - f_c)\} = \max\{0, X - Y - \rho\}.$$

If $\rho = 0$, then $e' = e$. If $\rho > 0$, then $X - Y \geq \rho > 0$ by the claim, so $e' + \rho = X - Y = e$. $\square$

PROOF OF LEMMA 5.1. By induction over reachable states. Initially every term is 0. Suppose (Q) holds for every account before an operation. The left side of $Q(a)$ depends on $o(a)$, on the backing a receives, on the backing it gives (through $c(a)$), on $\lambda(a)$ and on a 's own defaults. The right side depends on $\ell(a)$, which is fixed, and on $d(a)$, which never decreases. It suffices to check, for each operation, the accounts whose left side can increase.

BORROW(v, x) raises $o(v)$ only. By the precondition and $\kappa \leq 1$,

$$o(v) + x \leq \text{Lim}(v) \leq G(v) - c(v) + \sigma^{\text{in}}(v) + v^{\text{in}}(v),$$

using $c(v) \leq G(v)$ from (5). Hence the new residual is at most $G(v) - c(v)$, and $Q(v)$ holds in the form (5). Disbursement leaves $o(v)$ unchanged, and cancellation lowers it.

REPAY lowers $o(v)$, and so $e(v)$, and raises $d(v)$. The payer's state is unaffected.

BACK(u, v, x) adds x_c to $c(u)$ and adds backing that v receives, which can only lower $e(v)$. Since $u \neq v$, the residual $e(u)$ is unchanged. If $x_c = 0$, nothing else changes. If $x_c > 0$, then $\varphi(u) > 0$, which forces $\delta(u) = 0$ because $G(u) = 0$ otherwise. By definition (4), $x_c \leq G(u) - c(u)$ and $o(u) + x_c \leq \text{Lim}(u)$. As for **BORROW**, the second inequality gives $o(u) - \sigma^{\text{in}}(u) - v^{\text{in}}(u) \leq G(u) - c(u) - x_c$, and the right side is nonnegative by the first. So $e(u) \leq G(u) - c(u) - x_c$, that is, $e(u) + (c(u) + x_c) \leq G(u)$.

CUT(u, v, y) lowers $c(u)$ or $s^c(u)$, which cannot hurt $Q(u)$, and lowers the backing v receives, which can raise $e(v)$. The cut is accepted only if afterwards $o(v) \leq \text{Lim}(v)$. If $\delta(v) = 0$, the argument for **BORROW** applies, since $c(v)$ is unchanged. If $\delta(v) = 1$, then $\text{Lim}(v) = 0$, so $o(v) = 0$ and $e(v) = 0$.

STAKE and **UNSTAKE** touch no term of (Q).

DEFAULT of a loan of v . For each backer u , Λ_u rises by b_u through $\lambda(u)$, and $c(u)$ falls by $v(u, v)$ (release) or by b_u (consumption), in either case by at least b_u . Since $u \neq v$, $e(u)$ is unchanged, so the left side of $Q(u)$ does not rise. For v , $c(v)$ is unchanged, Λ_v rises by ρ , and $e(v)$ falls by at least ρ by Lemma 5.2. Setting $\delta(v) = 1$ does not affect (Q), which is stated in terms of $\ell + d$ and not of G . After its default, v can neither borrow nor commit credit, since $\text{Lim}(v) = \varphi(v) = 0$.

FUND, **RELEASE** and the pool's own operations touch no term of (Q). $\square$

THEOREM 5.3 (LOSS BOUND). *With issued lines fixed, every reachable state satisfies*

$$\Lambda + U \leq \sum_{a \in \mathcal{A}} (\ell(a) + d(a)).$$

PROOF. For every real X and every $Y \geq 0$, $\max\{0, X\} \leq \max\{0, X - Y\} + Y$. With $o^{\text{act}} \leq o$,

$$U \leq \sum_v \max\{0, o(v) - \sigma^{\text{in}}(v)\} \leq \sum_v (e(v) + v^{\text{in}}(v)) = \sum_a (e(a) + c(a)),$$

because each unit of unsecured backing is received by one account and committed by another. Adding $\Lambda = \sum_a \Lambda_a$ and summing (Q) over the finitely many accounts with nonzero state gives the claim. $\square$

The bound is attained. An account that pays dues d , then borrows its whole granted credit $\ell + d$ and defaults, realises a loss of exactly $\ell + d$. Section 8.2 reports how close random executions come. The bound is also independent of the backing graph: honest accounts may back one another in any pattern without changing it.

The bound localises to any coalition. For $S \subseteq \mathcal{A}$, let Λ^S be the realised loss on defaults of accounts in S , U^S the part of U owed by accounts in S , and $K_{H \rightarrow S}$ the total unsecured backing that accounts in $H = \mathcal{A} \setminus S$ have ever committed to accounts in S , counting each BACK once.

COROLLARY 5.4 (COALITIONS). *For every $S \subseteq \mathcal{A}$, in every reachable state,*

$$\Lambda^S + U^S \leq \sum_{a \in S} (\ell(a) + d(a)) + K_{H \rightarrow S}.$$

Honest backers lose at most what they committed to S : their burned credit is at most $K_{H \rightarrow S}$ and their slashed stake at most the stake they committed to S .

PROOF. Let $\lambda_S(u)$ be the credit charged to u at defaults of accounts in S . Then Λ^S is the sum of the residuals at those defaults and of $\lambda_S(u)$ over all backers u . The residuals and the charges to backers in S are part of $\sum_{a \in S} \Lambda_a$, so $\Lambda^S \leq \sum_{a \in S} \Lambda_a + \sum_{u \in H} \lambda_S(u)$. As in the proof of Theorem 5.3,

$$U^S \leq \sum_{v \in S} (e(v) + v^{\text{in}}(v)) \leq \sum_{a \in S} (e(a) + c(a)) + \sum_{u \in H, v \in S} v(u, v),$$

since the unsecured backing that accounts in S receive from S is at most what accounts in S have committed. Summing (Q) over S ,

$$\Lambda^S + U^S \leq \sum_{a \in S} (\ell(a) + d(a)) + \sum_{u \in H} \lambda_S(u) + \sum_{u \in H, v \in S} v(u, v).$$

Each unit of unsecured backing from H to S is, at any time, still committed, consumed by a charge, or released, and never more than one of these. So the last two sums together are at most $K_{H \rightarrow S}$. The statement about honest backers follows in the same way. $\square$

Accounts with no issued line, no dues and no backing from outside contribute nothing to the bound, however many there are. The bound is the capacity of a cut: the credit the issuer placed inside S , the dues S itself paid, and the unsecured trust that crosses into S . It parallels the role of attack edges in SybilLimit [Yu et al. 2008].

The last corollary separates what the reserve pays from what lenders bear. Let F be the capital added by FUND, D the amount released to lenders, and Λ_L the part of Λ that the reserve did not pay. Dues are the reserve's only other inflow, so

$$R = \sum_a d(a) + F - (\Lambda - \Lambda_L) - D. \quad (6)$$

COROLLARY 5.5 (LENDERS BEAR AT MOST THE ISSUED LINES). *With issued lines fixed, every reachable state satisfies*

$$\Lambda_L + \max\{0, U - R\} \leq \sum_a \ell(a).$$

PROOF. The release rule keeps $R \geq \sum_a d(a)$ after every release, so by (6) $D \leq F - (\Lambda - \Lambda_L) \leq F$ at every release. Since D changes only at releases and F never decreases, $D \leq F$ always. Substituting (6) into Theorem 5.3 gives

$$\Lambda_L + U - R \leq \sum_a \ell(a) - F + D \leq \sum_a \ell(a). \quad (7)$$

This is the claim when $U \geq R$. Otherwise we must show $\Lambda_L \leq \sum_a \ell(a)$. If the reserve has never been exhausted, $\Lambda_L = 0$. If it has, Λ_L last grew at a default that left $R = 0$. In that state (7) with $U \geq 0$ gives $\Lambda_L \leq \sum_a \ell(a)$, and Λ_L has not changed since. $\square$

Dues therefore cost lenders nothing: the losses that credit earned through dues can cause are absorbed by the reserve those dues funded. Lenders' exposure is to the issued lines only, and every unit of external first-loss capital F comes off it while losses exceed the reserve.

6 LINES THAT CHANGE

An issuer revises lines, and the bounds must survive revisions. Lowering a line is subtle: it cuts the account's capacity at once, but it does not undo loans already made against the old line.

Definition 6.1 (Idle times and peak lines). An account a is *idle* in a state where $o(a) = 0$ and $c(a) = 0$. Its *peak line* at time t is $\ell^*(a) = \max\{\ell_\tau(a) : \tau_a \leq \tau \leq t\}$, where τ_a is the last time up to t at which a was idle (the start of the execution if there is none).

THEOREM 6.2 (CHANGING LINES). *If the issuer may change lines at any time, every reachable state satisfies*

$$U \leq \sum_{a \in \mathcal{A}} (\ell^*(a) + d(a)).$$

PROOF. Let Λ_a^{cyc} be the part of Λ_a incurred since τ_a , and $\lambda^{\text{cyc}}(a) \leq \lambda(a)$ the corresponding part of $\lambda(a)$. We show that every account satisfies

$$e(a) + c(a) + \Lambda_a^{\text{cyc}} \leq \ell^*(a) + d(a). \quad (\text{Q}^*)$$

At τ_a the left side is 0. Between idle times, $\text{ISSUE}(a, \cdot)$ can only raise $\ell^*(a)$. Repayment, default, charges, stake, backing that a receives, cuts of backing from a , and cuts of backing into a once $\delta(a) = 1$ do not depend on $G(a)$, and they preserve (Q^*) by the arguments of Lemma 5.1. The remaining operations are BORROW by a , BACK from a and CUT into a while $\delta(a) = 0$, so that $\Lambda_a^{\text{cyc}} = \lambda^{\text{cyc}}(a)$. If $G(a) > 0$ and $G(a) \geq c(a)$, the arguments of Lemma 5.1 give $e(a) + c(a) \leq G(a)$ afterwards, and

$$G(a) + \lambda^{\text{cyc}}(a) = \ell(a) + d(a) - \lambda(a) + \lambda^{\text{cyc}}(a) \leq \ell^*(a) + d(a).$$

Otherwise $\max\{0, G(a) - c(a)\} = 0$, either because $G(a) = 0$ or because a lowered line has made $G(a) < c(a)$. Then $\varphi(a) = 0$, so BACK commits no credit, and BORROW and CUT leave $e(a) \leq \max\{0, G(a) - c(a)\} = 0$; the left side does not rise. Summing (Q^*) as in Theorem 5.3 bounds U . $\square$

Theorem 6.2 bounds the potential loss at every moment, but not the cumulative realised loss: Λ^{cyc} restarts at every idle time, so a line can be lost once per cycle of use. The peak line cannot be replaced by the current line.

PROPOSITION 6.3 (ROTATION). *For every $L > 0$ and $k \geq 1$ there is an execution in which the sum of current lines never exceeds L but $U = kL$, and in which every account then defaults, so that $\Lambda = kL$.*

PROOF. For $i = 1, \dots, k$: issue a line L to a new account a_i , let a_i borrow and draw L , and lower its line to 0. Each a_i keeps its loan, so after step i the potential loss is iL , while the sum of current lines is L during step i and 0 between steps. Each $\ell^*(a_i) = L$, so Theorem 6.2 is tight here. $\square$

Figure 3 plots the construction. It is the attack available to an issuer whose key or data feed is compromised, and it determines how an on-chain issuance budget must be charged. Suppose lines are issued as scores $q(a) \in [0, 1]$ times a fixed maximum M , so $\ell(a) = Mq(a)$. The budget keeps a *held* amount $\beta(a)$ per account. A report that sets $q(a)$ above $\beta(a)$ raises $\beta(a)$ to $q(a)$. A report is rejected if it raises $\sum_a \beta(a)$ above a budget B , or by more than a per-report cap. A release operation, callable by anyone, lowers $\beta(a)$ to $q(a)$, but only while a is idle.

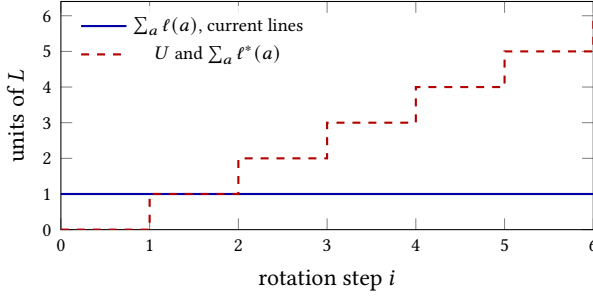


Fig. 3. Rotating one line of size L through k accounts (Proposition 6.3). The sum of current lines stays at L while the potential loss grows with every step. The sum of peak lines tracks it exactly.

COROLLARY 6.4 (ISSUANCE BUDGET). *If every line is issued through such a budget, then in every reachable state $U \leq MB + \sum_a d(a)$.*

PROOF. $\beta(a)$ is at least every score a has held since its last release, and releases occur only at idle times, so $\beta(a) \geq q^*(a)$, the peak score since τ_a . Hence $\sum_a \ell^*(a) \leq M \sum_a \beta(a) \leq MB$, and Theorem 6.2 applies. $\square$

A compromised issuer can misallocate its budget but cannot place more than the budget at risk at any time. Realised losses can recur once per cycle of use, which is what governance responses (a pause of new lending, a delay on changes to the issuer) and issuer capital are for. A budget charged on current scores would not bound U at all, by Proposition 6.3.

7 CREDIT FROM REPAYMENT HISTORY

The bounds place all risk on the right-hand side: issued lines and dues. A natural wish is to let accounts earn credit from their repayment history without an issuer. This section shows how much they may earn.

7.1 Farming

Consider a variant of the protocol in which (1) is replaced by $G(a) = (1 - \delta(a)) \max\{0, \ell(a) + \eta(H_a) - \lambda(a)\}$, where H_a is the record of a 's loans and repayments and $\eta \geq 0$ is a *history rule*. The protocol of Section 3 is the rule $\eta(H_a) = d(a)$.

Definition 7.1 (Recyclable histories). A history H is *recyclable with capital K* if an agent that holds K in liquid funds and controls a new account a , and possibly other accounts, can act so that afterwards a 's record is H , a has no open loans and no backing given or received, and the agent again holds $K - \pi(H)$ for some $\pi(H) \geq 0$. We call $\pi(H)$ the value H *surrendered*: what the history moved out of the agent's reach, net of anything the agent recovers in any capacity.

THEOREM 7.2 (FARMING). *Suppose accounts are free and some history H is recyclable with capital K and has $\eta(H) > \pi(H)$. Then for every $n \geq 1$, an agent holding $K + n\pi(H)$ can end with a net profit of $n(\eta(H) - \pi(H))$, which the other participants lose, provided the pool can lend $n\eta(H)$.*

PROOF. For $i = 1, \dots, n$, the agent creates a new account a_i and completes H on it, recovering $K - \pi(H)$, and tops its capital up to K . Each a_i then has $\ell = \lambda = \delta = 0$, no commitments and granted credit $\eta(H)$, so $\text{Lim}(a_i) \geq \eta(H)$. Each a_i borrows $\eta(H)$ and never repays. The agent has paid $n\pi(H)$ and received $n\eta(H)$. Abandoning an account costs the agent nothing, because an account is all that a default can sanction. $\square$

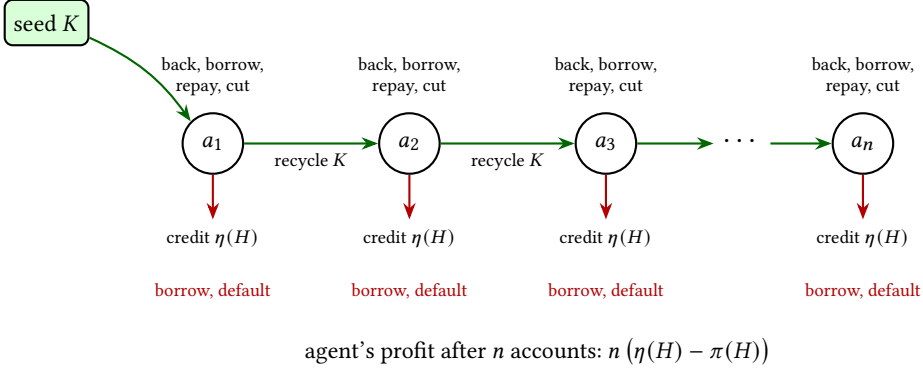


Fig. 4. The construction of Theorem 7.2 in this protocol. One seed of stake backs each new account in turn, is never at risk, and is recycled. Each account keeps the history credit $\eta(H)$, borrows it and defaults.

In this protocol, secured backing makes every history of fully repaid loans recyclable (Figure 4). The agent stakes K from a second account and backs a with it as secured backing. Then a borrows at most K and repays principal and interest, the backing is cut (allowed, because a owes nothing) and the stake is withdrawn. The stake is never slashed, and the history costs at most its interest. A rule that grants credit for repaid *principal* is therefore farmable at almost no cost. For example, the rule “capacity rises by 25% of each repaid principal, up to the maximum line” reaches the maximum after four loans. Repaid within the 24-hour grace period those loans pay no interest, so $\pi(H) = 0$ and each account yields the maximum line (Section 8.4). The same holds if an issuer grants lines for repayment counts: the policy, not where it runs, is what is farmable.

COROLLARY 7.3 (TRILEMMA). *No lending protocol has all three of the following properties:*

- (1) *accounts are free;*
- (2) *some history recyclable with finite capital earns more credit than it surrenders, $\eta(H) > \pi(H)$;*
- (3) *for every set S of accounts, the loss to others from loans to S is at most the lines issued to S , plus the value S has surrendered, plus the backing other accounts committed to S .*

PROOF. Let S be all the accounts of the agent in the construction of Theorem 7.2, including any that supplies the capital. Then S holds no issued lines, receives no backing from other accounts, and has surrendered $n\pi(H)$, but the loans to S cost others $n\eta(H) > n\pi(H)$. $\square$

This protocol keeps the first and third properties: by Corollary 5.4, with dues in place of surrendered value, as long as dues are surrendered (Section 7.2). Credit from history beyond that requires an identity cost or an issuer. If an identity costs k , a profit-seeking agent gains nothing from history credit up to $\pi(H) + k$ per identity. That is an incentive bound, not a loss bound: an agent that does not seek profit can still cost lenders k per identity. Credit beyond dues is therefore a decision someone must answer for. Theorem 5.3 charges it to the issuer’s lines, and Corollary 6.4 bounds it. This is the on-chain form of the result of Friedman and Resnick [2001] that, with cheap pseudonyms, newcomers must pay dues, and that the cost disappears only with identities issued once per person [Borge et al. 2017].

7.2 What counts as surrendered

Interest seems a natural measure of π , but an agent may also be a lender. Suppose an agent holds a share $s \in [0, 1]$ of the pool throughout, pays interest I on the loans of its accounts, and withdraws

before they default. Interest to lenders raises the value of all shares at once, while the reserve is a junior claim that is excluded from the value of shares.

PROPOSITION 7.4 (A BORROWER WHO IS ALSO A LENDER). *In this setting the agent recovers $s(1 - f - r)I$ through its shares, so $\pi(H) = I(1 - s(1 - f - r))$.*

- (1) *Under the rule that credits interest net of the fee, $\eta(H) = (1 - f)I$, the profit per account is $I(s(1 - f - r) - f)$, positive whenever $s > f/(1 - f - r)$, and for every $s > 0$ when $f = 0$.*
- (2) *Under the dues rule, $\eta(H) = rI$, the profit per account is $I(r - 1 + s(1 - f - r)) \leq -fI \leq 0$ for every $s \in [0, 1]$.*

PROOF. Of the interest I , the fee fI leaves the pool, the dues rI enter the reserve, and $(1 - f - r)I$ raises the value of the shares, of which the agent holds s . The profit is $\eta(H) - \pi(H)$. For the second rule, $s(1 - f - r) \leq 1 - f - r$. $\square$

The first rule was an earlier version of this protocol; simulation found the attack (Section 8.4). Under the dues rule, the dues stay in the reserve, which absorbs the very default that the credit they grant can cause. One channel remains, because the reserve is pooled: it also absorbs other borrowers' defaults.

PROPOSITION 7.5 (POOLED RESERVE). *In the setting of Proposition 7.4 under the dues rule, suppose that between the payment of the dues and the agent's withdrawal, other borrowers' defaults consume the dues rI from the reserve. Compared with the same agent without the history, the history changes the agent's wealth by $I(r - 1 + s(1 - f))$, which is positive only if $s > (1 - r)/(1 - f)$.*

PROOF. Relative to the agent without the history, the dues absorb losses that would otherwise have lowered the value of shares, of which the agent holds s ; this adds $s r I$ to the profit in Proposition 7.4. $\square$

The release rule of Section 3, which keeps all dues ever paid in the reserve, closes the other way the reserve could shrink, a release by the owner. What remains is a transfer under stress, available only to an agent that holds more than $(1 - r)/(1 - f)$ of the pool (70% at $r = 0.3$, $f = 0$) while other borrowers default. With π measured this way, the dues rule satisfies $\eta \leq \pi$ except in that case.

8 IMPLEMENTATION AND VALIDATION

8.1 Implementation

The protocol is implemented as a Solidity contract of about 24 kB of bytecode [Microcredit Protocol Project 2026]. Appendix B maps the symbols of Section 3 to the contract's state and functions. The implementation adds a lending pool with shares, interest at a fixed rate set at origination, meta-transactions, provisioning of overdue loans, and an issuer contract that enforces the budget of Corollary 6.4. A deployment on a public test network holds test funds only.

8.2 Stateful fuzzing

The bounds were checked against the implementation by stateful fuzzing in the style of property-based testing [Claessen and Hughes 2000], using the Foundry framework. A handler drives eleven accounts: three with issued lines (scores 1.0, 0.6 and 0.25 of a 100 USDC maximum), two that stake, and six Sybil accounts with nothing. It calls 21 operations in random order with random arguments: every borrowing path, repayment, default, backing and cutting, staking, provisioning, deposits, queued withdrawals, funding and release of the reserve, and time steps. The fee share is 10% and the reserve share 20%. The handler keeps an independent model of every loan and charge. Thirteen properties are asserted, each after every call of its own campaign: Theorem 4.1, the per-account

Table 1. One stateful fuzzing campaign (commit b725a85). Counts are totals over all runs; a default can slash stake, charge credit and leave a residual at once.

Property tests $\times$ runs per test $\times$ calls per run	$13 \times 1,300 \times 256$
Calls executed	4,326,400
Loans originated	260,511
Defaults	118,935
that slashed stake	27,447
that charged backers' credit	16,793
that left a residual to the reserve or lenders	78,111
Loans to Sybil accounts	52,109
Backing attempts by Sybil accounts accepted / rejected	1 / 2,311,491
Largest use of the per-account invariant (Q)	100.00%
Largest use of the loss bound, Theorem 5.3	99.98%
Assertion failures	0

invariant (Q), Theorem 5.3, Corollary 5.5, that Sybil accounts gain nothing and commit nothing they did not pay, the pool's solvency and accounting, and agreement between the contract and the model.

Table 1 summarises one campaign at the commit cited above: 13 property tests, each with 1,300 runs of 256 calls. No assertion failed. Random executions used up to 100.00% of an account's budget in (Q) and up to 99.98% of the bound of Theorem 5.3, so the bounds are close to tight on random executions, not only in constructed ones. The one backing by a Sybil account that was accepted committed credit the account had earned as dues, which the property on Sybil commitments allows and checks.

8.3 Mutation analysis

To test whether the validation detects violations, we seeded fourteen faults into the contract one at a time and reran the campaign at its default size, 64 runs of 80 calls per property, under three fixed fuzz seeds [Jia and Harman 2011]. Each fault breaks one rule that the proofs rely on. Table 2 lists them, how many of the three seeds detected each, and the properties that failed. A fault that no seed detected was run against the contract's unit tests. Fuzzing detected eleven of the fourteen faults under every seed. The unit tests detected the remaining three. The three faults the fuzzing missed each remove a precondition that the handler rarely or never crosses: it never asks an account to back itself, its model of releasing the reserve predates the rule that keeps dues, and a cut that leaves a borrower owing more than its limit needs a cut and an open loan drawing on the same backing. Strengthening the handler on these three points is the next step for the harness.

8.4 Attack simulation

A discrete simulation that reuses the contract's integer arithmetic compares four mechanisms under six attacks. The mechanisms are personalised PageRank over attestations (the predecessor of this protocol), the 25% rule of Section 7 on top of this protocol, this protocol with credit for interest net of the fee (an earlier version), and this protocol as implemented. The attacker controls n accounts, brings in any capital it needs, and defaults on every loan it does not repay. Table 3 reports its net profit, the change in its wealth, at $n = 1$ and $n = 64$ (fee 10%, reserve 30%), and Figure 5 plots three attacks against n .

Each alternative fails to the attack its design invites, with profit linear in n . PageRank fails to rings rooted in deposits that are withdrawn after the scores are computed (5,818 USDC at $n = 64$).

Table 2. Seeded faults, the number of fuzz seeds (of three) under which the default campaign detected each, and the properties or unit tests that failed.

Seeded fault	Seeds	Detected by
Coverage ignored: unsecured backing always counts in full	3/3	Thm. 4.1, coverage
A default does not burn the unsecured backers' credit	3/3	Lem. 5.1, credit ledgers
A default does not slash the secured backers' stake	3/3	credit ledgers
Free credit ignores the backer's own loans	3/3	Lem. 5.1
Received backing can be passed on	3/3	Sybil commitments, Lem. 5.1
Backing with credit does not record the commitment	3/3	Thm. 4.1, coverage, Thm. 5.3, Cor. 5.5, credit ledgers, model agreement
Backing can be cut below what the borrower owes	0/3	unit tests: testBackerRepayingOneLoanFreesNoMoreThanItPaid, testBackingCannotBeCutBelowWhatTheBorrowerOwes
Backing is released at a default while the borrower has other open loans	3/3	Cor. 5.5, Lem. 5.1
A defaulted account keeps its granted credit	3/3	coverage, credit ledgers
A new loan is checked against the limit, not the unused limit	3/3	Thm. 5.3, Cor. 5.5, Lem. 5.1
Dues are credited on interest net of the fee, not the reserve share	3/3	Sybil gain, Lem. 5.1, credit ledgers
The owner may release dues from the reserve	0/3	unit test: testOnlyCapitalBeyondDuesIsReleased
Committed stake can be withdrawn	3/3	Thm. 4.1, provisions, credit ledgers, pool ledgers
An account may back itself	0/3	unit test: testBackMetaRejectsSelfBackingAndMoreThanTheBackerHas

Table 3. Attacker net profit in USDC with 1 and 64 accounts. “n/a”: the attack needs a feature the mechanism lacks.

Attack	PageRank		25% rule		Net-interest dues		This protocol	
	$n = 1$	$n = 64$	$n = 1$	$n = 64$	$n = 1$	$n = 64$	$n = 1$	$n = 64$
Ring of new accounts	0.00	304.76	0.00	0.00	0.00	0.00	0.00	0.00
Ring on withdrawn deposits	0.00	5,818.18	0.00	0.00	0.00	0.00	0.00	0.00
Recycled stake, no interest	n/a	n/a	100.00	6,400.00	0.00	0.00	0.00	0.00
Recycled stake, 30-day loans	n/a	n/a	95.58	6,117.03	-0.31	-19.84	-2.15	-137.89
Colluding backer, line 100	189.50	885.96	100.00	100.00	100.00	100.00	100.00	100.00
Borrower who lends, $s = 0.5$	n/a	n/a	18.47	1,182.02	1.87	119.42	-3.73	-238.85

The 25% rule fails to one recycled seed (6,400). Credit for interest net of the fee fails to a borrower who is also a lender, as Proposition 7.4 predicts: the simulated profit per account, 1.87, equals $I(s(1 - f - r) - f)$ with $I = 9.33$, $s = 0.5$, $f = 0.1$, $r = 0.3$. Against the implemented protocol, no attack gains from additional accounts. The one profitable row, a backer that colludes with the attacker using its own line of 100, gains that line and no more, which Theorem 5.3 charges to the issuer that granted it. Measured on a copy of the deployed contract, the stress case of Proposition 7.5

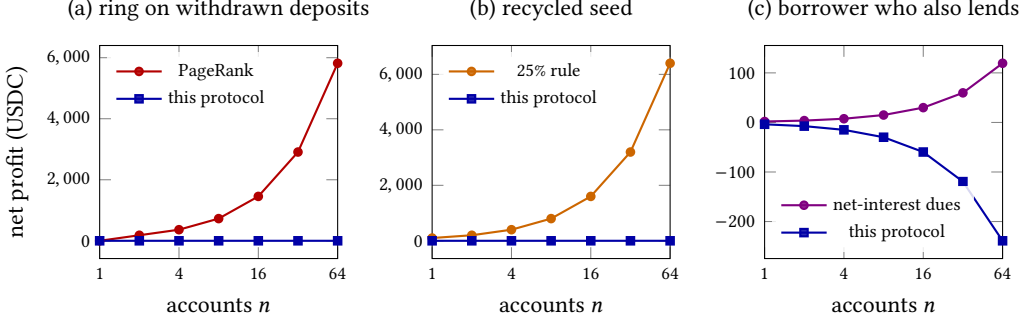


Fig. 5. Attacker net profit against the number of accounts, for the attack that defeats each alternative. Against this protocol the profit is 0 or negative for every n .

matched its formula: with $r = 0.3$, $f = 0$ and interest $I = 2.33$, an agent holding half the pool ended 0.47 behind a passive lender and one holding 91% ended 0.49 ahead, that is, $I(s - 0.7)$.

9 DISCUSSION

Where the risk goes. Theorem 5.3 does not say that nothing can go wrong. It says where an attacker must go. Rings, wash trades and new accounts yield nothing. An honest backer can be persuaded or deceived into committing credit to an attacker, and loses at most what it committed (Corollary 5.4). The issuer can be fooled or compromised, and the damage is bounded by its budget (Corollary 6.4). Governance, the key that sets the issuer and the maximum line, sits outside the model; in the implementation it is a multisignature account behind a timelock, with a guardian that can only pause new lending. The protocol’s security thus reduces to issuance and governance, the only places where credit is created.

Probability of default versus loss given default. Burning a backer’s credit recovers no cash, so unsecured backing does not lower the loss given default. It can lower the probability of default through selection and monitoring [Ghatak and Guinnane 1999; Stiglitz 1990], but whether it does is an empirical question [Giné and Karlan 2014]. The bounds hold either way; only stake and the reserve recover cash.

One hop. Received backing cannot be passed on. In a credit network, borrowing capacity is a maximum flow over paths of any length [Dandekar et al. 2011]. Allowing longer paths raises liquidity, but it charges accounts for borrowers they reach only through intermediaries they trust, and it enlarges the cut in Corollary 5.4, since an edge from an account without credit would then carry the credit of accounts behind it. The one-hop rule keeps every charge on an account that chose the borrower. Quantifying the liquidity this costs is the subject of a companion paper.

Open problems. Several questions remain open, and we invite work on them.

- (1) *Issuer capital.* Corollary 6.4 bounds an issuer’s exposure, but lenders bear it. A mechanism in which the losses Theorem 5.3 attributes to an issuer’s lines are charged first to capital the issuer posts, with the budget a multiple of that capital, would make the issuer a delegated monitor with a stake in its own judgement [Diamond 1984]. What multiple is safe, given that realised losses recur once per cycle of use?

- (2) *Equilibrium backing*. The model takes backing decisions as given. In a game in which backers choose whom to back and how much to monitor, and default probabilities respond, which equilibria exist, and does the coverage discount affect them?
- (3) *Optional relaying*. Can multi-hop capacity be offered to sources that opt in per line, with a bound that interpolates between Corollary 5.4 and the max-flow bound of credit networks?
- (4) *Pooled versus earmarked reserves*. Earmarking the reserve behind dues in use removes the transfer of Proposition 7.5 but stops that part of the reserve from absorbing other losses. What is the optimal rule?

10 CONCLUSION

Collateral-free lending with free identities is possible if borrowing capacity is conserved rather than inferred. In the protocol studied here, every unit of capacity is a liability of a named account. Lenders' loss is then bounded by the issued lines plus the dues paid, in every reachable state and for any coalition, independently of the number of accounts. Histories can earn credit only up to what they surrender, and larger credit requires an issuer whose budget is charged on peak lines. The results move the hard problem from detecting Sybil accounts, which is impossible, to issuing credit well, which is a problem of information and accountability.

ACKNOWLEDGMENTS

HermesCRBot (hermes-agent-909), an AI agent working with the project, attacked every version of the protocol. Its ring attacks on the PageRank design led to the design studied here, its proposal to grow capacity with repaid principal led to Theorem 7.2, and it measured the stress case reported in Section 8.4 on a copy of the deployed contract.

Disclosure of AI authorship. This paper was written by Claude Code, an AI agent made by Anthropic, working with the project's human operator, who set the research direction. The proofs, the implementation, the fuzzing campaign, the mutation analysis and the simulation were carried out by AI agents. All of them are public and reproducible.

Data and code availability. The contract, its tests and the simulation, at commit b725a85, are at <https://github.com/scottonchain/microcredit-contract>. The paper's sources, the scripts that extract its data, and the fuzzing and mutation records are at <https://github.com/scottonchain/microcredit-theory>.

REFERENCES

- Aave. 2020. Aave Protocol Whitepaper, version 1.0. White paper.
- Timothy Besley and Stephen Coate. 1995. Group Lending, Repayment Incentives and Social Collateral. *Journal of Development Economics* 46, 1 (1995), 1–18.
- Maria Borge, Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, and Bryan Ford. 2017. Proof-of-Personhood: Redemocratizing Permissionless Cryptocurrencies. In *IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 23–26.
- Jeremy Bulow and Kenneth Rogoff. 1989. Sovereign Debt: Is to Forgive to Forget? *American Economic Review* 79, 1 (1989), 43–50.
- Alice Cheng and Eric Friedman. 2005. Sybilproof Reputation Mechanisms. In *Proceedings of the ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON)*. ACM, 128–132.
- Alice Cheng and Eric Friedman. 2006. Manipulability of PageRank under Sybil Strategies. In *First Workshop on the Economics of Networked Systems (NetEcon)*.
- Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 268–279.
- Pranav Dandekar, Ashish Goel, Ramesh Govindan, and Ian Post. 2011. Liquidity in Credit Networks: A Little Trust Goes a Long Way. In *Proceedings of the 12th ACM Conference on Electronic Commerce (EC)*. ACM, 147–156.

- Dimitri do B. DeFigueiredo and Earl T. Barr. 2005. TrustDavis: A Non-Exploitable Online Reputation System. In *Seventh IEEE International Conference on E-Commerce Technology (CEC)*. IEEE, 274–283.
- Douglas W. Diamond. 1984. Financial Intermediation and Delegated Monitoring. *Review of Economic Studies* 51, 3 (1984), 393–414.
- John R. Douceur. 2002. The Sybil Attack. In *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 (Lecture Notes in Computer Science, Vol. 2429)*. Springer, 251–260.
- Eric Friedman, Paul Resnick, and Rahul Sami. 2007. Manipulation-Resistant Reputation Systems. In *Algorithmic Game Theory*, Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani (Eds.). Cambridge University Press, Chapter 27.
- Eric J. Friedman and Paul Resnick. 2001. The Social Cost of Cheap Pseudonyms. *Journal of Economics & Management Strategy* 10, 2 (2001), 173–199.
- Maitreesh Ghatak and Timothy W. Guinnane. 1999. The Economics of Lending with Joint Liability: Theory and Practice. *Journal of Development Economics* 60, 1 (1999), 195–228.
- Arpita Ghosh, Mohammad Mahdian, Daniel M. Reeves, David M. Pennock, and Ryan Fugger. 2007. Mechanism Design on Trust Networks. In *Internet and Network Economics (WINE 2007) (Lecture Notes in Computer Science, Vol. 4858)*. Springer, 257–268.
- Xavier Giné and Dean S. Karlan. 2014. Group versus Individual Liability: Short and Long Term Evidence from Philippine Microcredit Lending Groups. *Journal of Development Economics* 107 (2014), 65–83.
- Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678.
- Dean Karlan, Markus Möbius, Tanya Rosenblat, and Adam Szeidl. 2009. Trust and Social Collateral. *Quarterly Journal of Economics* 124, 3 (2009), 1307–1361.
- Robert Leshner and Geoffrey Hayes. 2019. Compound: The Money Market Protocol. White paper, version 1.0.
- Microcredit Protocol Project. 2026. microcredit-contract: Solidity implementation, tests and analysis. <https://github.com/scottonchain/microcredit-contract>. Commit b725a85. The credit model is documented in docs/CREDIT_MODEL.md.
- Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *Financial Cryptography and Data Security (FC 2021) (Lecture Notes in Computer Science, Vol. 12674)*. Springer.
- Geoffrey Ramseyer, Ashish Goel, and David Mazières. 2020. Liquidity in Credit Networks with Constrained Agents. In *Proceedings of The Web Conference (WWW)*. ACM.
- Paul Resnick and Rahul Sami. 2009. Sybilproof Transitive Trust Protocols. In *Proceedings of the 10th ACM Conference on Electronic Commerce (EC)*. ACM, 345–354.
- Joseph E. Stiglitz. 1990. Peer Monitoring and Credit Markets. *World Bank Economic Review* 4, 3 (1990), 351–366.
- Orfeas Stefanos Thyfronitis Litos and Dionysis Zindros. 2017. Trust Is Risk: A Decentralized Financial Trust Platform. In *Financial Cryptography and Data Security (FC 2017) (Lecture Notes in Computer Science, Vol. 10322)*. Springer, 340–356.
- Haifeng Yu, Phillip B. Gibbons, Michael Kaminsky, and Feng Xiao. 2008. SybilLimit: A Near-Optimal Social Network Defense against Sybil Attacks. In *IEEE Symposium on Security and Privacy*. IEEE, 3–17.
- Haifeng Yu, Michael Kaminsky, Phillip B. Gibbons, and Abraham Flaxman. 2006. SybilGuard: Defending against Sybil Attacks via Social Networks. In *Proceedings of ACM SIGCOMM*. ACM, 267–278.

A NOTATION

B CORRESPONDENCE WITH THE IMPLEMENTATION

Table 4. Notation.

Symbol	Meaning
$\ell(a), \ell^*(a)$	issued line; peak line since a was last idle
$d(a)$	dues: share r of the interest on a 's loans, paid into the reserve
$\lambda(a)$	credit burned when borrowers backed by a defaulted
$\delta(a)$	1 once a has defaulted on a loan of its own
$G(a)$	granted credit, equation (1)
$s(a), s^c(a)$	stake; stake committed to backing
$c(a)$	granted credit committed to backing
$\sigma(u, v), v(u, v)$	secured and unsecured backing from u to v
$\sigma^{\text{in}}(v), v^{\text{in}}(v)$	secured and unsecured backing v receives
$\kappa(u)$	coverage of u 's unsecured commitments, equation (2)
$\text{Lim}(v)$	borrowing limit, equation (3)
$\varphi(u)$	free credit, equation (4)
$o(a), o^{\text{act}}(a)$	open principal; its disbursed part
$e(a)$	residual exposure, $\max\{0, o(a) - \sigma^{\text{in}}(a) - v^{\text{in}}(a)\}$
Λ, Λ_a	realised loss; its part attributed to a
U	potential loss: disbursed principal not covered by secured backing
R, F, D	reserve balance; external capital added; amount released
f, r	fee share and reserve share of interest
η, π	history rule; value surrendered by a history

Table 5. Symbols and the state or functions of the contract DecentralizedMicrocredit that hold or compute them.

Symbol or operation	Contract
$\ell(a)$	<code>getCreditScore(a) × maxLoanAmount / 10⁶</code>
$d(a), \lambda(a), \delta(a)$	<code>duesPaid, creditLoss, defaultedLoans</code>
$G(a)$	<code>grantedCredit</code>
$s(a), s^c(a), c(a)$	<code>stakeOf, stakeCommitted, creditCommitted</code>
$\sigma(u, v), v(u, v)$	<code>getBacking(u, v)</code>
$\text{Lim}(v), \varphi(u)$	<code>getBorrowLimit, getFreeCredit</code>
$\kappa(u)$	<code>inside _backingReceived</code>
BORROW	<code>requestLoan, borrowAndDisburseMeta (via _originateLoan)</code>
REPAY	<code>repayLoan, repayWithPermit, repayLoanMeta (via _repay)</code>
BACK, CUT	<code>back, backMeta (via _setBacking)</code>
DEFAULT	<code>markDefaulted, _chargeBackers</code>
FUND, RELEASE	<code>fundReserve, releaseReserve</code>
Issuance budget	<code>OracleScoreProvider: budgetHeld, releaseBudget</code>